

MEDIATEK

MT8676 Hypervisor ISP Driver Introduction

Version: 1.0
Release date: 2025-07-11

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2025-07-11	宋清华/汪锦航	正式版本

目录

版本记录 2

目录 3

图目录 3

1 ISP Driver 概述 5

2 Camsys (P1) 6

2.1 架构 6

2.1.1 Camsys Kernel Driver 6

2.1.2 Camsys Userspace 7

2.1 Camsys Media Device 拓扑结构 8

2.2 Camsys Userspace 控制流程 9

2.3 Camsys Debug 9

2.3.1 Camsys 关键 Log 9

2.3.2 Camsys 常见问题 10

2.3.3 Camsys 常见 Debug 指令 12

3 Imgsys(P2) 14

3.1 架构 14

3.2 Imgsys 用户空间 14

3.3 Imgsys 内核空间 17

3.4 Imgsys Debug 18

3.4.1 Imgsys 关键 Log 18

3.4.2 Imgsys 常见问题 19

附件一 附加条款 21

图目录

图 1-1. Camera 架构概述 5

图 2-1. Camsys 架构图 6

图 2-2. 简化版 Camsys 媒体拓扑 8

图 2-3. PipeMgr 控制流程 9

图 3-1. Imgsys 软件架构 14

图 3-2. Imgsys enqueue/dequeue 流程 15

图 3-3. User 使用 imgstream 流程 16

图 3-4. Imgsys V4L2 topology.....17

1 ISP Driver 概述

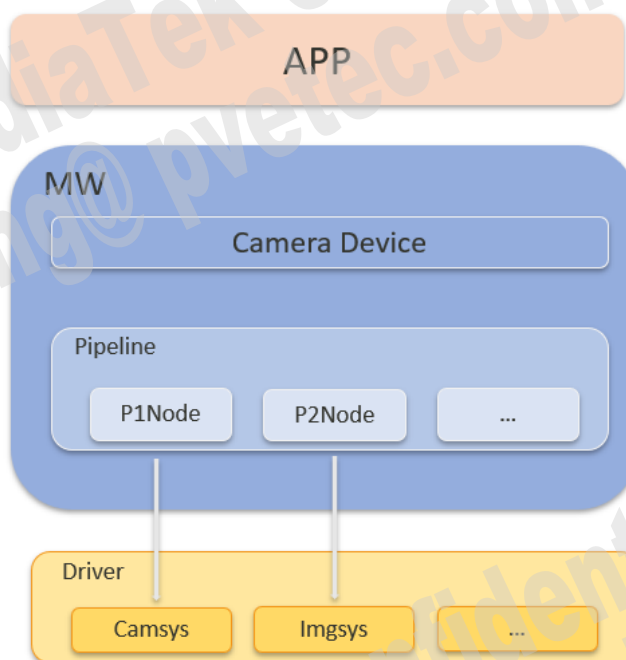


图 1-1. Camera 架构概述

MW core level 包含很多的 Imagenode 与底层 driver 进行交互，与 ISP 相关的 Imagenode，我们根据功能把它们区分为 P1Node (pass1 node) 和 P2Node (pass2 node)。

- Camsys 作为 pipeline 中 P1Node 的驱动组件，负责从 Sensor 接收图像，并主要具备 Raw 域的图像处理能力。
- Imgsys 作为 pipeline 中 P2Node 的驱动组件，负责对从 P1 接收到的图像进行进一步处理，能够在 YUV 域对图像进行降噪和缩放等操作。

2 Camsys (P1)

2.1 架构

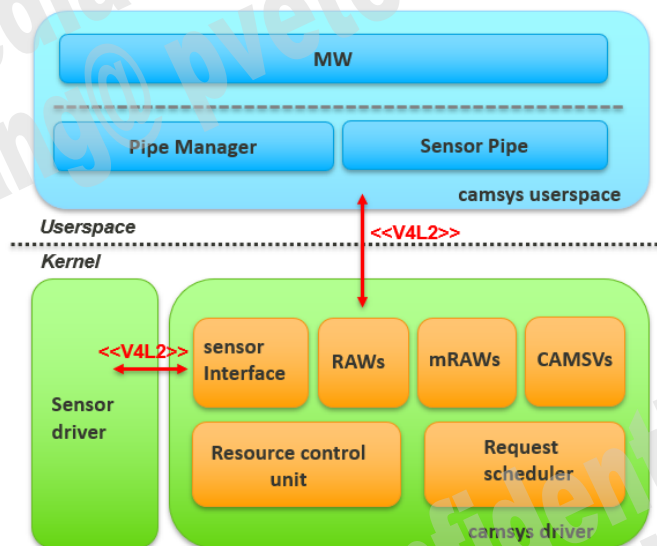


图 2-1. Camsys 架构图

2.1.1 Camsys Kernel Driver

Camsys Kernel Driver 是为 MediaTek 平台上的摄像头系统硬件组件设计的内核级驱动。它基于 V4L2 和 Media Framework 框架实现，并整合了 Linux Kernel Upstream 驱动，向 userspace 提供了标准的 V4L2 接口，确保在不同平台下驱动的兼容性。

Platform Drivers

每个 Camsys 硬件组件都被设计为独立的 Platforma Driver，包括 sensor interface、raw、mraw (miniraw) 和 camsv (用来收图的 HW，不对图像进行处理，例如用来收 YUV sensor 数据)。这种模块化设计便于灵活、可扩展地管理和维护各个组件，确保硬件控制的稳定与高效。

- **整合 Linux Kernel Upstream 驱动:** 为了保证 Camsys 内核驱动与 Linux Kernel 其他组件的交互，camsys 驱动实现了以下功能：
 - **电源管理:** 通过 device-link 和运行时电源管理 (PM_runtime)，实现摄像头系统的高效、可靠电源控制。
 - **内存访问:** 基于 DMA 框架，支持内存设备操作，包括 SMMU 集成和 SMI 配置，确保数据传输的高效与安全。
 - **Media Framework 和 V4L2:** 基于 Media Framework 和 V4L2 接口实现 Camsys 驱动，允许上层 MiddleWare 通过这些标准接口与摄像头系统交互。
- **标准驱动与用户接口:** Camsys 内核驱动为媒体操作提供了一套标准的驱动和用户接口：

- 与厂商 Sensor 驱动协同工作，实现从硬件到应用层的无缝集成。
- 为应用层提供统一接口，便于开发者开发和集成媒体相关应用。

2.1.2 Camsys Userspace

为了便于管理 camera pipeline 并控制从 Sensor 出图，Camsys Userspace 部分主要提供以下功能：

- 简化 V4L2 操作
- Pipeline 创建
- 设定图像格式
- 控制 Streaming

Camsys Userspace 主要包括两部分：

- **SensorPipe：**
 - 一对一关系：每个 Sensor 对应一个 SensorPipe.
 - 流控制：控制 Sensor Stream ON/OFF
 - 读写数据：SensorPipe 负责图像和统计值 enqueue 和 deque.
 - 事件轮询：支持 Start of Frame (SOF)、ESD Recovery 等事件的轮询，便于同步与时序控制。
- **PipeMgr：**
 - 查找设备：发现 camsys 内部设备及 Camsys 驱动的拓扑结构
 - 拓扑建立：通过 V4L2 Media Framework 建立 pipeline 的拓扑结构，用于处理和读取图像数据。
 - 简化应用控制：简化 V4L2 应用控制流程，提升开发效率。

2.1 Camsys Media Device 拓扑结构

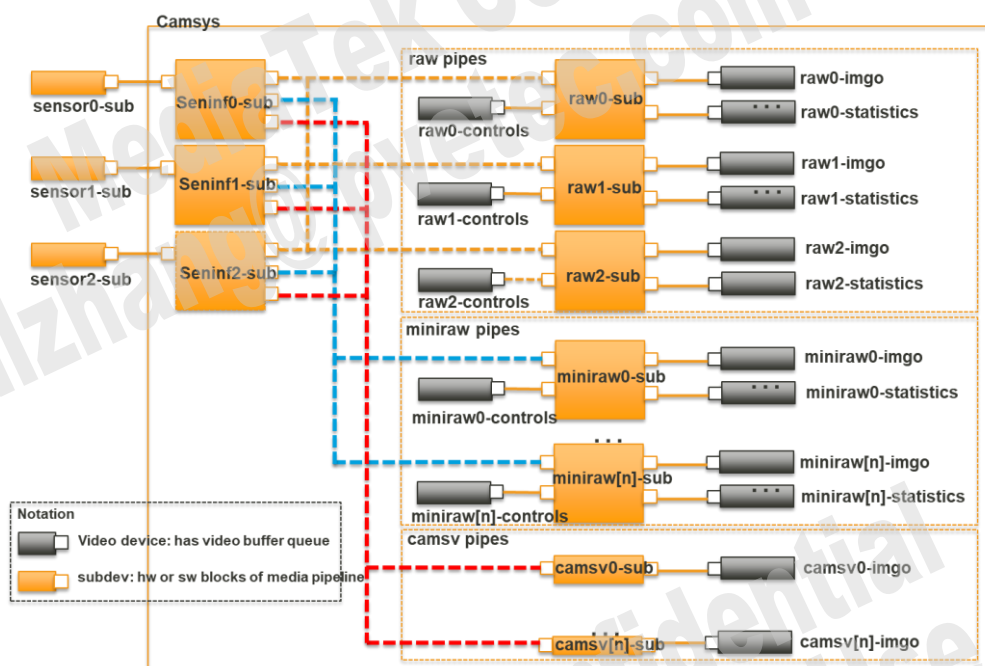


图 2-2. 简化版 Camsys 媒体拓扑

Camsys 媒体拓扑是 Camsys 框架下 camera pipeline 的结构化表达。它由多个子设备组成，包括 sensor、sensor interface、sub-device 和 Video Device。这些组件通过硬件允许的数据流链路互联，简化了 ISP 的控制流程。

- **Sub-Devices:** 在 Camsys Topology 中，每个 sub-device 在图像数据处理流程中承担特定任务：
 - Sensor Interface：为 Sensor 及各类 Camsys 图像处理引擎提供接口，是 Sensor 与处理单元之间的桥梁，负责原始图像数据的传递。
 - 图像处理引擎：包括 raw engine、camsv engine 和 mraw engine，各自具备不同的处理能力。
- **Video Device:** 映射 DMA Port 口，用于硬件读写数据
 - 图像：每个 DMA Port 作为一个 Video Device，读写图像数据，并可进行格式设定，确保每路视频流可按需配置像素格式和分辨率。
 - 统计值：用于采集和处理统计数据，主要用于 3A（自动曝光、自动对焦、自动白平衡）算法。
- **Link:** 用于 sub-device 和 video device 之间的连接。
 - 创建：链路由驱动自动创建，反映硬件允许的数据流路径。
 - 配置：用户可根据应用需求启用或禁用已创建的链路，实现灵活的数据流配置。

2.2 Camsys Userspace 控制流程

Camsys userspace 控制流程是基于标准 V4L2 应用控制流程实现，为简化操作，Camsys 用户空间提供了如图 2-3 所示的控制流程：

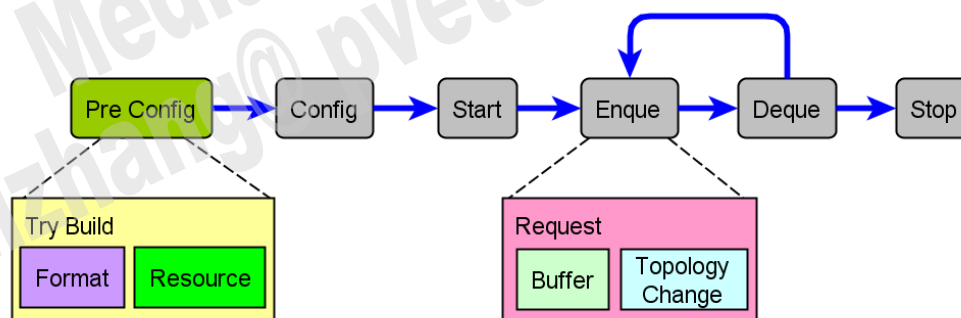


图 2-3. PipeMgr 控制流程

Pipemgr 关键控制流程说明：

1. Pre Config

- 查询 ISP 能力
- 根据上层带的格式和图像大小构建 camsys 的 Topology

2. Config

- 设定 pre config 阶段的 topology

3. Start

- 开启 camsys

4. Enque/Deque

- 图像/统计值
- Sensor 设定
- 切换控制

5. Stop

- stream off sensor 和关闭 camsys

2.3 Camsys Debug

2.3.1 Camsys 关键 Log

User space:

1. start:

```
06-24 04:39:27.636 1148 1343 D MtkCam/PipeMgr: [streamOn] node: |80[mtk-cam camsv-0
main-stream]|83[mtk-cam camsv-0 main2-stream]|81[mtk-cam camsv-0 ext-stream]|82[mtk-cam
camsv-0 main1-stream]|84[mtk-cam camsv-0 main3-stream] requestBuf(1)
```

2. enqueue:

```
06-24 04:39:27.642 1148 4290 D MtkCam/SensorPipe: [enqueue] [2]sensorId(15)_m02
jobId(0)_(-2|0) requestFd(617)_released portSel(0) fs(-) nodeIds( 1:32 ) {scene(NORMAL)
maxExpNum(0) expNum(0) expOrder(SL) type(NONE) } SensorCtrl{exp_cnt(0) flk(0) fps(0)
test_ptn(-1,0_-1/-1/-1/-1) SensorScenario(-1) extend(-) awbGain(0) master(0)} mtk-cam
camsv-0 timeout:19000ms BufInfo{ 32-0_616 } 222
```

3. deque:

```
06-24 04:39:27.718 1148 4424 D MtkCam/SensorPipe: [deque] sensorId(15) jobId(0)_(-2|0)
requestFd(617) state(DONE) poll(1) hwDebugStatus(0x0) nodeIds( 1:32(NO_ERROR):1 )
```

4. sof event:

```
06-24 04:39:27.692 1148 4447 D MtkCam/SensorPipe: [waitSof] sensorId(15) frame sync event(0) sof(1) dummy(0)
```

5. stop:

```
06-24 04:39:27.772 1148 1148 D MtkCam/PipeMgr: [streamOff] node: |80[mtk-cam camsv-0
main-stream]|83[mtk-cam camsv-0 main2-stream]|81[mtk-cam camsv-0 ext-stream]|82[mtk-cam
camsv-0 main1-stream]|84[mtk-cam camsv-0 main3-stream] releasebuffer(1)
```

Kernel space:

1. CTX stream on:

```
[ 8625.435751] CTMThread8-00: mtk-cam
1a000000.camisp:[name:mtk_cam_isp7sp&]mtk_cam_ctx_stream_on:ctx_1
```

2. CSI stream on:

```
[ 8625.441963] camsv_worker-0: seninf 1a00e000.seninf-top:seninf-csi-port-14:
[name:mtk_cam_isp7sp&][seninf_csi_s_stream] enable(1)
```

3. CTX stream off:

```
[ 8625.474812] binder:1118_3: mtk-cam 1a000000.camisp:
[name:mtk_cam_isp7sp&]mtk_cam_ctx_engine_off: ctx-0 pipe 0x8 engine 0x10
```

4. CSI stream off:

```
[ 8625.475195] binder:1118_3: seninf 1a00e000.seninf-top:seninf-csi-port-2:
[name:mtk_cam_isp7sp&][seninf_csi_s_stream] enable(0)
```

2.3.2 Camsys 常见问题

vsync timeout

kernel log:

```
06-21 14:15:51.786 <6>[ 1200.191375][T609892] kworker/5:2: mtk-cam 1a000000.camisp:
[name:mtk_cam_isp7sp&]mtk_cam_watchdog_monitor_vsync: vsync may timeout, last ts =
1198702038899
06-21 14:15:55.883 <6>[ 1204.287673][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-6: [name:mtk_cam_isp7sp&]clk-fmeter-isp(416000) clk-fmeter-csi0(26000)
clk-fmeter-csi1(0) clk-fmeter-csi2(0) clk-fmeter-csi3(312000) clk-fmeter-csi4(0) clk-
fmeter-csi5(0)
```

```

06-21 14:15:55.883 <6>[ 1204.287688][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-6:
[name:mtk_cam_isp7sp&]MipiRx_ANA6:CDPHY_RX_ANA_SETTING_1:(0x04340244),CDPHY_RX_ANA_0/_1/_2
/_3/_4/_5/_6/_7/_8:(0x000747d3)/(0x00420844)/(0x44000b0b)/(0x02000c0c)/(0x01010b0b)/(0x700
00044)/(0x04090909)/(0x04090909)/(0x203f0400)
06-21 14:15:55.883 <6>[ 1204.287699][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-6:
[name:mtk_cam_isp7sp&]MipiRx_ANA6:CDPHY_RX_ANA_AD_0/_1:(0x33)/(0x0),AD_HS_0/_1/_2:(0x0)/(0
x0)/(0x0)
06-21 14:15:55.883 <6>[ 1204.287712][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-6:
[name:mtk_cam_isp7sp&]MipiRx_ANA7:CDPHY_RX_ANA_SETTING_1:(0x05530200),CDPHY_RX_ANA_0/_1/_2
/_3/_4/_5/_6/_7/_8:(0x000707d3)/(0x00420844)/(0x44000b0b)/(0x02000b0b)/(0x01010303)/(0x700
00044)/(0x04090909)/(0x04090909)/(0x203f0400)
06-21 14:15:55.883 <6>[ 1204.287768][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-6:
[name:mtk_cam_isp7sp&]MipiRx_ANA7:CDPHY_RX_ANA_AD_0/_1:(0x33)/(0x0),AD_HS_0/_1/_2:(0x0)/(0
x0)/(0x0)
06-21 14:15:55.883 <6>[ 1204.287781][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-6:
[name:mtk_cam_isp7sp&]Csi3_Dphy_Top:LANE_EN/_SELECT:(0xf01)/(0x80403102),CLK_LANE0_HS/1_HS
:(0x10090000)/(0x10090000),DATA_LANE0_HS/1_HS/2_HS/3_HS:(0x30800400)/(0x30800400)/(0x30800
400)/(0x30800400),DPHY_RX_SPARE0:(0xf1)
06-21 14:15:55.883 <6>[ 1204.287793][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-6:
[name:mtk_cam_isp7sp&]Csi3_Dphy_Top:DPHY_RX_DESKEW_CTRL/_TIMING_CTRL/_LANE0_CTRL/_LANE1_CT
RL/_LANE2_CTRL/_LANE3_CTRL:(0x3f000904)/(0x00030000)/(0x00700000)/(0x00700000)/(0x00700000
)/(0x00700000)
06-21 14:15:55.883 <6>[ 1204.287802][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-6:
[name:mtk_cam_isp7sp&]Csi3_Dphy_Top:DPHY_RX_DESKEW_IRQ_EN/_CLR/_STATUS:(0x80000000)/(0x000
00000)/(0x00000000),DPHY_RX_IRQ_EN/_STATUS:(0x0000000f)/(0x00000000)
06-21 14:15:55.883 <6>[ 1204.287809][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-6:
[name:mtk_cam_isp7sp&]Csi3_Cphy_Top:CPHY_RX_CTRL:(0x0),CPHY_RX_DETECT_CTRL_POST:(0x1249241
0),CPHY_RX_IRQ_EN/_STATUS:(0x80000100)/(0x00000000)
06-21 14:15:55.883 <6>[ 1204.287818][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-6:
[name:mtk_cam_isp7sp&]TOP_CTRL2:(0x8),TOP_MUX_CTRL_0/_1/_2/_3/_4/_5:(0x3020106)/(0x7060504
)/(0xb0a0908)/(0xf0e0d0c)/(0x13121110)/(0x1514),debug_vb:1,debug_ft:7
06-21 14:15:55.883 <6>[ 1204.287827][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-6: [name:mtk_cam_isp7sp&]before clear cam mux0,recSize = 0x0,irq =
0x0|0x0
06-21 14:15:55.883 <6>[ 1204.287836][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-6:
[name:mtk_cam_isp7sp&]SENINF6_CSI2_PRBS_EN/_OPT:(0x0)/(0x1f0000),CSIRX_MAC_CSI2_EN/_OPT/_I
RQ_STATUS/_MULTI_ERR_F_STATUS:(0xf)/(0xb6)/(0x0)/(0x0),SENINF6_CSI2_IRQ_STATUS:(0x0),CSIRX_
MAC_CSI2_RESYNC_MERGE_CTRL:(0x106)
06-21 14:15:55.883 <6>[ 1204.287845][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-6: [name:mtk_cam_isp7sp&]total_delay:0ms/7ms,SENINF6_PkCnt:(0x0),ret=0
06-21 14:15:56.274 <6>[ 1204.679029][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-9: [name:mtk_cam_isp7sp&]total_delay:7ms/7ms,SENINF6_PkCnt:(0x0),ret=-
1

```

```

06-21 14:15:56.274 <6>[ 1204.679051][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-9: [name:mtk_cam_isp7sp&][sensor9][mtk_cam_seninf_debug]
CSI_MAC6_CSI2_IRQ_STATUS/_MULTI_ERR_F_STATUS:(0x0)/(0x0),SENINF_CSI2_IRQ_STATUS:(0x0),C/DP
HY_RX_IRQ_STATUS:(0x0)/(0x0)
06-21 14:15:56.274 <6>[ 1204.679061][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-9:
[name:mtk_cam_isp7sp&]*SENINF0_MUX_CTRL0/_CTRL1/_IRQ_STATUS/_MUX_SIZE/_ERR_SIZE/_EXP_SIZE:
(0x1)/(0x1f0308)/(0x0)/(0x0)/(0x0)/(0x0)
06-21 14:15:56.274 <6>[ 1204.679075][T615359] kworker/5:5: seninf 1a00e000.seninf-
top:seninf-csi-port-9:
[name:mtk_cam_isp7sp&]cam_mux_3_CTRL/RES/ERR/OPT/IRQ:(0x8380)/(0x0)/(0x5a20f00)/(0x0)/(0x2
1009e83)/(0x0),tag03_vc/_dt(0x83000000/0x9e000000),tag47_vc/_dt(0x0/0x0)

```

以上 log 是发生 vsync timeout 后，Kernel log 里面关于 seninf dump status log，出现 vsync timeout 表示在一段时间里面，camsys 没有收到 vsync irq，意味着 camsys 无法正常出图，发生这种问题，通常检查上面 log 红色部分：

1. csi irq status[keyword: **IRQ_STATUS**], 正常 csi irq status 为 0x324 结尾，表示有 FS/FE, ECC 校验正确，上面 log csi irq status 为 0，表示没有收到任何数据；
2. 如果 csi irq status 不正常，
 - a) 需要再检查 csi clk [keyword: **clk-fmeter-csi** (使用哪个 csi port，对应看哪个 clk)]，正常是大于等于 312M，上面 log csi clk 正常；
 - b) 如果是 DPHY，需要检查是否有 SOT Error [keyword: **DPHY_RX_IRQ_EN/_STATUS**]，如果 status 非 0，表示有 SOT Error 出现，可能 clk 和 data 没有正常识别，常见原因是没有检测到 Lpstatus，需要 sensor 端检查信号是否符合 MIPI-DPHY 规范；
 - c) 如果上面 csi clk 正常并且没有 SOT Error，说明 sensor 没有数据或者数据异常，需要进一步分析 sensor 端；
3. 如果 csi irq status 正常，
 - a) 看 seninf 后端 cam mux 是否有正常按照 vc/dt 分离出数据[keyword: **recSize =**]，recSize 应该等于每一路 vc/dt 的数据大小，后面 irq 正常为 0x100，如果这里没有数据，需要检查对应 sensor 端 vc/dt 是否有数据；
 - b) 如果 cam mux 数据也正常，但仍然有 vsync timeout error 出现，则需要进一步分析，常见情况有系统 performance 太差，导致 vsync irq 没有被及时处理，camsys 的 watchdog 也会出现 vsync timeout error。

2.3.3 Camsys 常见 Debug 指令

查看 seninf status:

```
adb shell cat /sys/devices/platform/soc/1a00e000.seninf-top/status
```

以下是 MT8676 seninf status dump:

```
clk-fmter-isp(416000) clk-fmter-csi0(0) clk-fmter-csi1(312000) clk-fmter-csi2(0) clk-fmter-csi3(0) clk-fmter-csi4(0) clk-fmter-csi5(0)
```

```
[seninf-14] port 0 intf 0 test 0 cphy 0 lanes 0
[seninf-13] port 0 intf 0 test 0 cphy 0 lanes 0
[seninf-12] port 0 intf 0 test 0 cphy 0 lanes 0
[seninf-11] port 0 intf 0 test 0 cphy 0 lanes 0
[seninf-10] port 3 intf 6 test 0 cphy 0 lanes 0
[seninf-3] port 3 intf 6 test 0 cphy 0 lanes 0
[seninf-0] port 3 intf 6 test 0 cphy 0 lanes 0
[seninf-9] port 4 intf 8 test 0 cphy 0 lanes 0
[seninf-8] port 4 intf 8 test 0 cphy 0 lanes 0
[seninf-7] port 4 intf 8 test 0 cphy 0 lanes 0
[seninf-6] port 4 intf 8 test 0 cphy 0 lanes 0
[seninf-5] port 5 intf 10 test 0 cphy 0 lanes 0
[seninf-4] port 4 intf 8 test 0 cphy 0 lanes 0
[seninf-2] port 2 intf 4 test 0 cphy 0 lanes 0
source sensor1 flags 0x4
[seninf-1] port 1 intf 2 test 0 cphy 0 lanes 4
source sensor0 flags 0x5
csirx_mac_dst irq_stat 0x003f0324 seninf_irq_stat 0x00000000
csi2_line_frame_num 0x00000000
csi2_packet_status 0x1680002c
csi2_packet_cnt_status 0x037707a8
rx-ana settle ck 0x00 dt 0x1a
rx-ana trail en 0 param 0x00
CSIRX_MAC_CSI2_RESYNC_MERGE_CTRL 0x00000106 CDPHY_RX_ANA_SETTING_1 0x04340244 DPHY_RX_SPARE0 0x000000f1
data_not_enough_cnt : <0>
err_lane_resync_cnt : <0>
crc_err_cnt : <0>
ecc_err_double_cnt : <0>
ecc_err_corrected_cnt : <0>
after write clear csi_irq 0x0 seninf_irq_stat 0x00000000
[0] vc 0x0 dt 0x2c mux 2 cam 0
mux[2] en 1 src 2 irq_stat 0xc
fifo_overrun_cnt : <0>
cam[0] en 1 src 16 exp 0x8700f00 res 0x8760f00 err 0x8760000 irq_stat 0x102
size_err_cnt : <0>
--- mux meter ---
width 3840 height 2166
h_valid 3409, h_blank 1254
v_valid 11088119, v_blank 2774510
mipi_pixel_rate 468458451
v_blank 6671 us
h_blank 3 us
line_time 11 us
```

seninf csi irq status, 以0x324结尾表示收图正常

vc&dt

预期收图大小和实际收图大小

seninf dbg log (打开 seninf dbg level log, 可以看到更详细的 seninf 模块 log)

```
adb shell "echo 1 > /sys/module/mtk_cam_isp7sp/parameters/seninf_dbg_log"
```

camsv dbg log (打开 camsv dbg level log, 可以看到更详细的 camsv 模块 log, 比如 fmt/size 设定等)

```
adb shell "echo 1 > /sys/module/mtk_cam_isp7sp/parameters/debug_cam_sv"
```

detailed dbg log (打开整个 camsys 详细 log, 可以看到每个 request 的运行情况以及 hw irq 的信息)

```
adb shell "echo 0xffffffff > /sys/module/mtk_cam_isp7sp/parameters/debug_opts"
```

pipemgr dbg log (打开 camsys userspace 部分详细 log, 可以看到详细的上层 V4L2 控制流程)

```
adb shell "setprop vendor.debug.pipemgr.loglevel 4"
```

3 Imgsys(P2)

3.1 架构

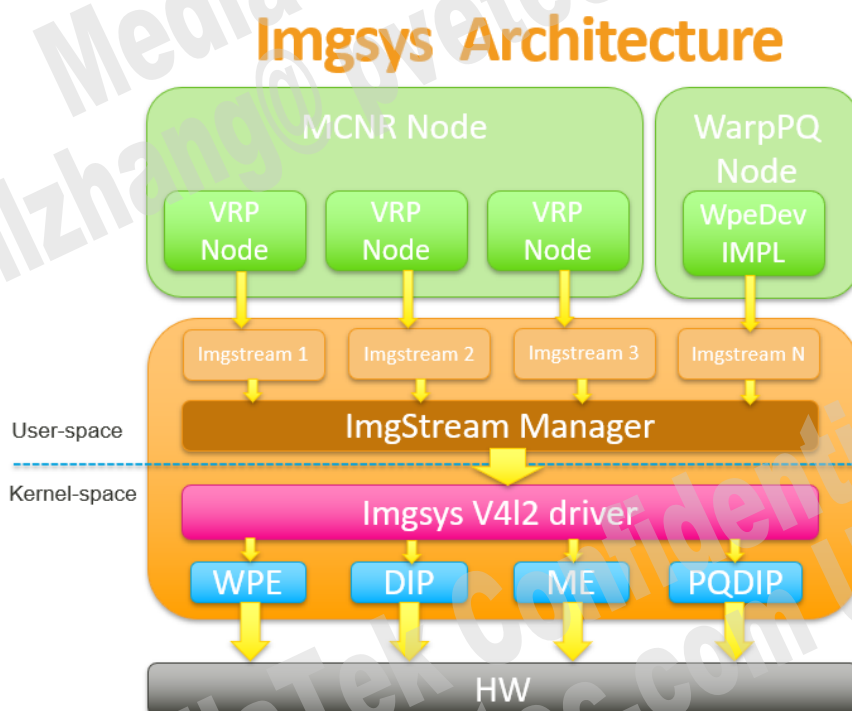


图 3-1. Imgsys 软件架构

imgsys 模块用于控制 P2 hardware(HW)，它的主要功能是进行 ISP 后处理。imgsys 模块包括两个部分：用户空间和内核空间，用户空间部分命名为 Imgstream，负责与 middleware(MW) 和内核通信。ImgStream 支持多用户，MW 可以创建多个 ImgStream 实例。内核空间部分负责控制 P2 硬件模块，包括用于缩放的 TRAW、用于降噪的 DIP 和用于调整大小、裁剪和旋转的 PQDIP。用户空间通过 ioctl 与内核空间通信。

3.2 Imgsys 用户空间

ImgStream 是硬件抽象层。它是 MW 和硬件之间的交互桥梁。ImgStream 负责为 MW 提供接口来控制硬件，V4L2 ioctl 并不适合 MW，因此 ImgStream 对此做了封装之后向上提供了同时处理不同硬件的友好界面。Imgsys 是一个 M2M（内存到内存）驱动程序，它通过使用 v4l2 框架负责处理多用户的需求。Imgsys 有许多硬件模块，包括 TRAW，DIP 和 PQDIP，用户可以根据设置同时使用硬件模块的独立和直连模式。用户可以通过使用 Imgstream 实现不同的硬件组合来做大小调整、旋转、翻转、降噪等操作。

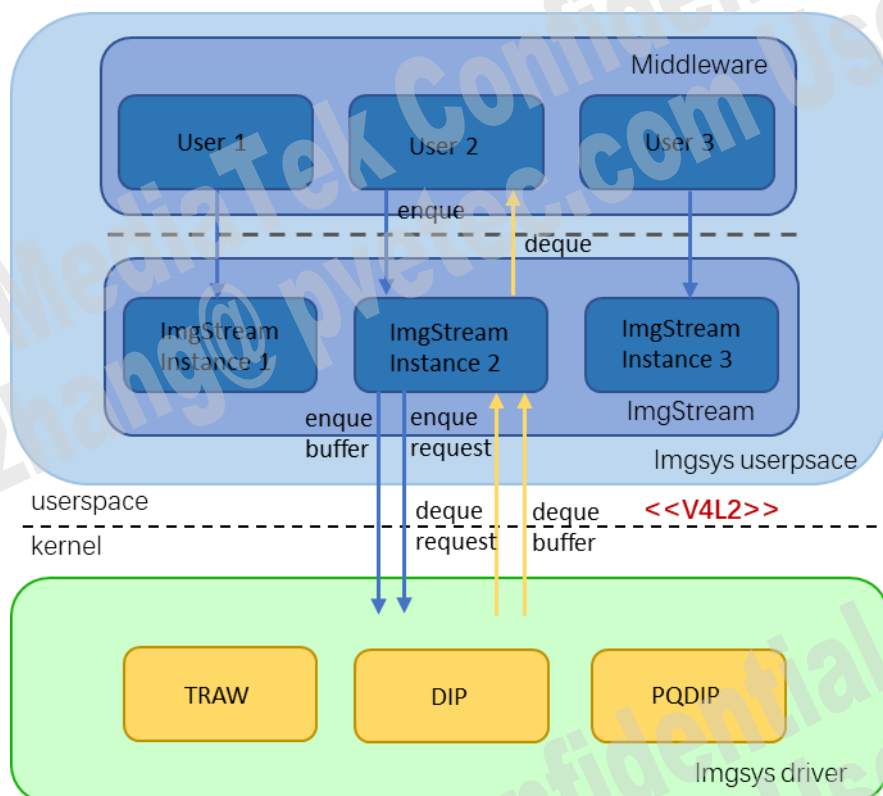


图 3-2. Imgsys enqueue/dequeue 流程

ImgStream 提供了 init, enqueue, dequeue, uninit 接口供 MW 调用来控制 imgsys 的 HW。详细介绍如下:

- Init 头文件:** ImgStream.h, 函数原型: virtual MBOOL init(const ImgInitParam& rUserParam);
 init 的接口 API 会 open video device 和 sub-device, 创建 frame callback thread 和 request callback thread。当第一个 user call imgstream 的 API 时, 就会去设置 HW 的 stream on。
- Enqueue 头文件:** ImgStream.h, 函数原型: virtual MBOOL enqueue(const std::shared_ptr<const ImgParams>& pParams);
 enqueue 的接口 API 会把 MW 传入的参数转为 Imgstream 内部的参数, Imgstream 首先 enqueue buffer 到 Kernel, Kernel 将会保存这些 enqueue 进来的 buffer。等到 Imgstream enqueue request 到 Kernel 的时候, Kernel 就会把 buffer 和 request 一起带给 driver 做 enqueue。
- Dequeue 头文件:** V4L2VideoDevice.h, 函数原型: int dequeueBuffer();
 dequeue 依赖 init 阶段创建的 frame callback thread 和 request callback thread, 监测从 Kernel 送到 user space 的 frame done callback and request done callback 信号。当 frame done callback thread 接收到信号时, 就会 call MW 注册给 Imgstream 的函数去通知 MW。当 request callback thread 接收到信号后, 会 call kernel 的 ioctl 去 dequeue buffer 和 request, 并且 call MW 注册的 callback 函数。

- Uninit 头文件: `ImgStream.h`, 函数原型: `virtual MBOOL uninit();`

当一个 `ImgStream` 的 user call `uninit` 接口 API 时, 会去 free 这个 user 申请的 resource buffer, 当最后一个 user call `uninit` 接口时, 会去设置 HW 的 stream off。

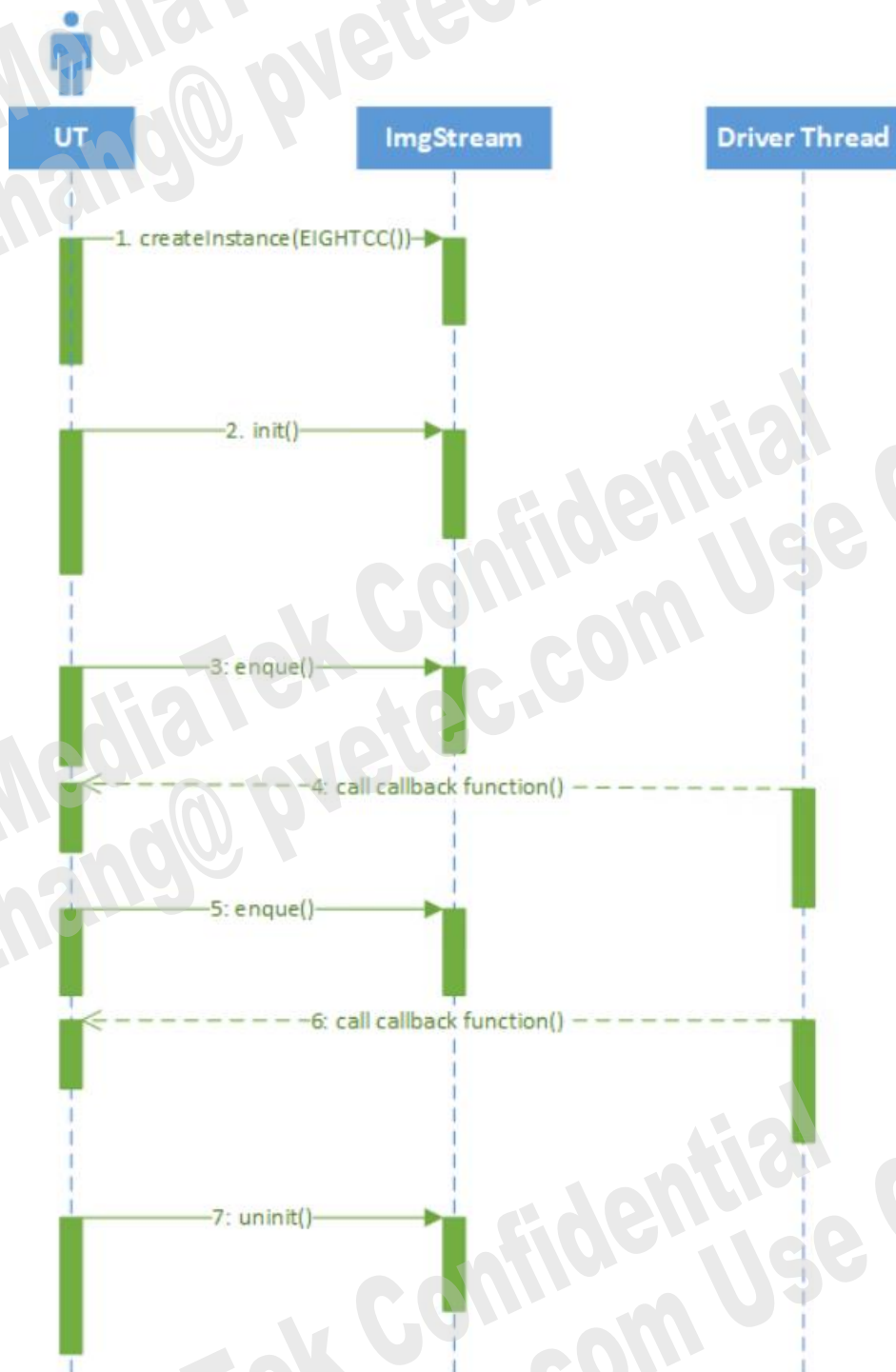


图 3-3. User 使用 `imgstream` 流程

Imgsys User 控制 API 流程如图所示，User 只需 call imgstream 提供的简易 API 接口，即可实现使用 imgsyst HW 做图像后处理的功能。

3.3 Imgsys 内核空间

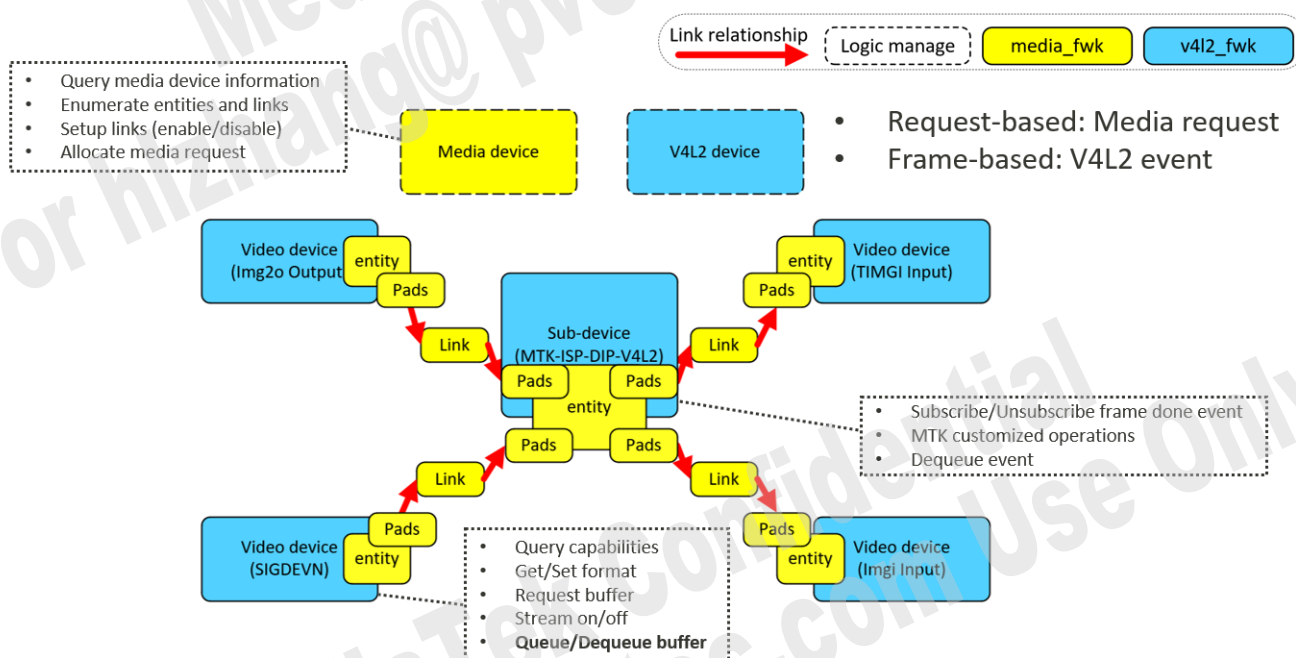


图 3-4. Imgsys V4L2 topology

Imgsys topology 描述了 Imgsys 框架的 pipeline 结构。它由一个 sub-device 和各种 video device 组成。topology 通过硬件之间的数据流的链路相互连接。

• Sub-Device

imgsys 架构包含一个名为 MTK-ISP-DIP-V4L2 的 sub-device，专为图像后处理任务而设计。该 sub-device 作为数字图像处理（DIP）硬件的接口，利用 V4L2 框架实现对图像处理操作的内核级控制。它支持一系列功能，包括色彩校正，降噪和图像增强，以产生卓越的图像质量。统一的 sub-device 简化了与用户空间应用程序的集成，简化了开发人员对于复杂 camera pipeline 的控制。

• Video Device

在 imgsys 架构中，每个 dma 端口是一个 video device，用于描述输入和输出端口，如 img2o、TIMGI、Imgi。由于 imgsys 中 dma 端口太多，用户空间单独控制每个 video device 会导致频繁调用 ioctl，MPIS 会比较高。因此设计了一个描述所有 dma 端口的 video device，命名为 SIGDEVN，它在每个队列中只被用户空间调用一次。

- **Links**

链路描述了媒体拓扑中子设备与视频设备之间的连接。在 imgsys 拓扑中，只有一个 sub-device 和几个 video device。在 probe 启动阶段，在 sub-device 和 video device 之间建立链路。

3.4 Imgsys Debug

3.4.1 Imgsys 关键 Log

Imgsys 在每个阶段会印出一些关键的 log，便于发生问题时的 debug。

- **Init**

```
ImgStream: [ImgStream::init] + mMemMode=0 userid=305f33707276(vrp3_0)
this:0xb400007d86b6fa00
{ .mMaxNumOfInflightRequests=6 .mSecTag=0 .mBatchNum=0 .mMaxFps=30 .mMaxInOutWidth=1280 .m
MaxInOutHeight=720 .mPriority=-19 .mLowLatency=0 .mInitDLWKBuf=0 .mIsCapture=0 }
mThisValue(0xb400007d86b6fa00)
```

- **Enque to Kernel**

```
ImgStreamMgr: [DECKEY][enqueue] Queue Request End(954us)
{ vrp3_0 .tid=1476,4424 .track=1000001000000000 } .request={ .requestFD=294 .state=2
#A=05-12 08:11:59.663925#B=05-12 08:11:59.663982#C=05-12 08:11:59.664004#D=05-12
08:11:59.664008#E=05-12 08:11:59.664920#F=05-12 08:11:59.463050#G=05-12
08:11:59.463129#H=05-12 08:11:59.463237 }
ImgParams={ .mRequestNo=0 .mFrameNo=50 .mvFrameParams[2]={ { .mvIn[3] .mvOut[1] .mSyncToke
nNotifyList={ 189 } .mSyncTokenWaitList={ 187 195 } }
{ .mvIn[6] .mvOut[3] .mSyncTokenNotifyList={ 194 } .mSyncTokenWaitList={ 189 } } } }
VNDescBuf={ .fd=267 .offset=4951424 .bufsize=309464 .bufva=7d4847dd80 }
CtrlMetaBuf={ .fd=266 .offset=436224 .bufsize=27264 .bufva=7d60e22800 }
this=b400007d86b6fa00
```

- **Callback from Kernel**

```
ImgStream: [DECKEY][handleRequestSuccess] request done
{ vrp3_0 .tid=1476,4424 .done=0 .track=1050500010b0000 .request={ .requestFD=294 .state=
0 #A=05-12 08:11:59.663925#B=05-12 08:11:59.663982#C=05-12 08:11:59.664004#D=05-12
08:11:59.664008#E=05-12 08:11:59.664920#F=05-12 08:11:59.666539#G=05-12
08:11:59.463129#H=05-12
08:11:59.463237 } .ImgParams={ .mRequestNo=0 .mFrameNo=50 .mvFrameParams[2]={ { .mvIn[3] .
mvOut[1] .mSyncTokenNotifyList={ 189 } .mSyncTokenWaitList={ 187 195 } }
{ .mvIn[6] .mvOut[3] .mSyncTokenNotifyList={ 194 } .mSyncTokenWaitList={ 189 } } } } }
```

- **Uninit**

```
ImgStream: [ImgStream::uninit] + userid=305f33707276(vrp3_0) this:0xb400007d86b6fa00
```

3.4.2 Imgsys 常见问题

下面列出 Imgsys 测试阶段遇到的比较常见的问题：

- **ImgStream Request Guard Timeout**

ImgStream request guard 是 imgsys 内部一个监测器，监测一个 request 从 enqueue 到 callback 执行结束的整个过程是否超时，目前一个 request 的超时时间设置为 4s, 超过这个时间就会触发 aee。

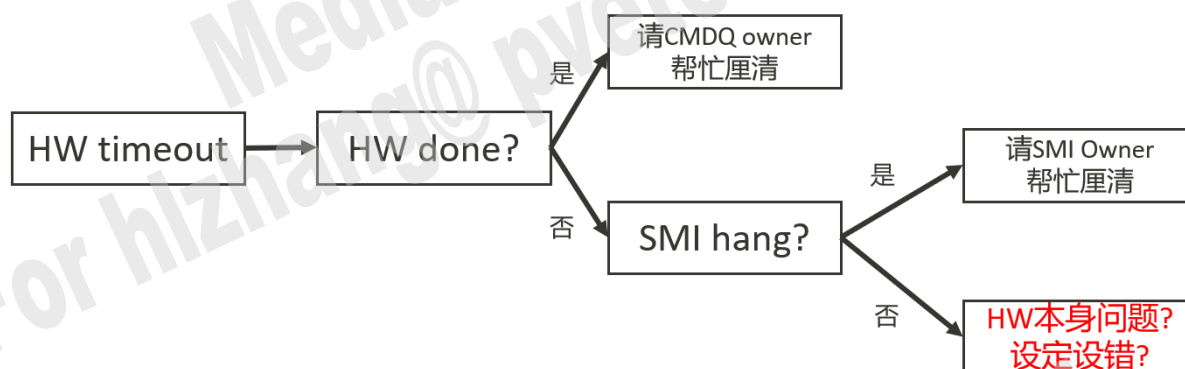
如果发生了 request timeout 要根据关键 log 分析当前超时的 request 卡在哪个阶段,再去重点看那个阶段的 log。

比如下面的例子就是卡在从 kernel deque 后，再往上层 MW callback 的阶段。

```
imgstream-TimeGuard: [RequestGuard] executed 13094 ms (>=4000) TIMEOUT; FORCE ABORT; {May
block on callback to wpepq66} track=0x1060600010b0000 suspect=User<wpepq66>
R={ wpepq66 .tid=31531,20081 .done=0 .track=1060600010b0000 .request={ .requestFD=897 .st
ate=0 #A=05-11 10:15:15.716984#B=05-11 10:15:15.743041#C=05-11 10:15:15.745507#D=05-11
10:15:15.745515#E=05-11 10:15:15.768573#F=05-11 10:15:15.805129#G=05-11
10:15:15.819603#H=05-11 10:15:15.742877 } }
(onTimeout) {#346:vendor/mediatek/proprietary/hardware/mtkcam-
core/hw/imgstream/src/utis/guard/RequestTimeGuard.cpp}
```

- **HW Timeout**

HW timeout 也会导致 ImgStream request guard timeout，这是卡在 Imgstream enqueue kernel 后没有在超时时间内 deque，Kernel log 会 dump 超时的 HW 的 register，从 dump 的 register 分析 HW timeout 的原因：



根据过往经验且与DE 讨论

- **Translation Fault**

Translation fault 是 iommu 通过虚拟地址转物理地址过程中发生了 error，Kernel log 会印出 SMMU/IOMMU

translation iova fail

```
[T400187] irq/349-arm-smm: ME_TranslationFault_callback: 0x34070060 3E77A580, 00000000, 00000000, 00000000
[T400187] irq/349-arm-smm: ME_TranslationFault_callback: 0x34070070 00000000, 00000000, 3E779F01, 3E779F00
[T400187] irq/349-arm-smm: ME_TranslationFault_callback: 0x34070080 00000000, 00000000, 00000000, 3E779800
[T400187] irq/349-arm-smm: ME_TranslationFault_callback: 0x34070090 3EF57C00, 3E39FF00, 3F59D8B0, 3E773800
[T400187] irq/349-arm-smm: ME_TranslationFault_callback: 0x340700A0 3E773700, 3E776940, 3E9972C0, 3E777390
```

解题过程:

1. 从 smmu/iommu log 确认是哪一个 HW 使用的哪一个 DMA port 口产生了 TF error;
2. 从 log 里找到出问题的 va 值, 确认最接近此 iova 的 buffer 为哪一块;
3. 定位 buffer 后寻找其异常点 (buffer 分配太小/HW 使用前提早释放)。

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。